

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5

8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5
9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5

Code of Conduct:

I Geferlin Boffy R 192571080 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

ANSWERS:

1. Complete Query Processing Pipeline

ALGORITHM Query_Processing_Pipeline(SQL_Query)

INPUT:

SQL_Query

OUTPUT:

Query_Result

BEGIN

Step 1: Receive SQL Query

Read SQL_Query from the user.

Step 2: Lexical Analysis

Convert SQL_Query into tokens.

Identify keywords, table names, column names,
operators, constants, and symbols.

Step 3: Parsing

Check whether the SQL syntax is correct.

Generate a Parse Tree / Syntax Tree.

IF syntax is invalid THEN

Display "Syntax Error"

STOP

END IF

Step 4: Semantic Analysis

Check whether tables and columns exist.

Check data types and user permissions.

IF semantic error occurs THEN

Display "Semantic Error"

STOP

END IF

Step 5: Query Translation

Convert the Parse Tree into a Relational Algebra
expression or logical query plan.

Step 6: Query Optimization

Generate alternative query execution plans.

Estimate the cost of each plan.

Select the plan with the lowest estimated cost.

Step 7: Physical Plan Generation

Convert the selected logical plan into physical
operations such as:

- Table Scan
- Index Scan
- Selection
- Join
- Sorting

Step 8: Query Execution

Send the selected execution plan to the execution engine.

Access required data pages from storage.

Perform the required operations.

Step 9: Result Generation

Produce the final result relation.

Step 10: Return Result

Display or return Query_Result to the user.

END

2. Cost-Based Query Optimization for Different Join Orderings

ALGORITHM Cost_Based_Join_Optimization

INPUT:

Relations R1, R2, ..., Rn

Statistics of each relation

Buffer pages B

OUTPUT:

Best_Join_Order

BEGIN

Generate all possible join orderings.

Best_Cost $\leftarrow$ INFINITY

Best_Join_Order $\leftarrow$ NULL

FOR each Join_Order DO

 Total_Cost $\leftarrow$ 0

 Current_Result $\leftarrow$ First relation in Join_Order

 FOR each remaining relation R DO

 Estimate number of pages in Current_Result.

 Estimate number of tuples in Current_Result.

 Estimate I/O cost of joining

 Current_Result with R.

 Example for Nested Loop Join:

 Join_Cost =

 Pages(Current_Result)

 +

 Pages(Current_Result) *

 Pages(R) / (B - 2)

 Total_Cost $\leftarrow$ Total_Cost + Join_Cost

 Estimate size of the new join result.

```

        Current_Result ← New Join Result

    END FOR

    IF Total_Cost < Best_Cost THEN
        Best_Cost ← Total_Cost
        Best_Join_Order ← Join_Order
    END IF

END FOR

RETURN Best_Join_Order, Best_Cost

END

```

Complexity Analysis:

The optimizer compares different join orders and selects the order having the minimum estimated I/O cost. For n relations, the number of possible left-deep join orders can be approximately $n!$.

3. Nested Loop Join Algorithm

ALGORITHM Nested_Loop_Join

INPUT:

```

    Relation R
    Relation S
    Join_Condition

```

OUTPUT:

```

    Join_Result

```

BEGIN

```

    Join_Result ← Empty

```

```

    FOR each tuple r IN R DO

```

```

        FOR each tuple s IN S DO

```

```

            IF Join_Condition(r, s) = TRUE THEN
                Add combined tuple (r, s) to Join_Result
            END IF

```

```

        END FOR

```

```

    END FOR

```

```

    RETURN Join_Result

```

END

Block Nested Loop Join

ALGORITHM Block_Nested_Loop_Join

INPUT:

```

    Relations R and S
    Number of buffer pages B

```

OUTPUT:

Join_Result

BEGIN

Join_Result $\leftarrow$ Empty

FOR each block of (B - 2) pages of R DO

Load the block of R into memory.

FOR each page of S DO

Load page of S into memory.

FOR each tuple r in R block DO

FOR each tuple s in S page DO

IF Join_Condition(r, s) = TRUE THEN

Add (r, s) to Join_Result

END IF

END FOR

END FOR

END FOR

END FOR

RETURN Join_Result

END

Time/I/O Complexity:

Let:

- bR = number of pages in relation R
- bS = number of pages in relation S
- B = number of available buffer pages

For Block Nested Loop Join:

Cost = bR + $\text{ceil}(bR / (B - 2)) \times bS$

In tuple comparison terms, the basic nested loop join can require approximately:

$O(|R| \times |S|)$

tuple comparisons.

4. Deadlock Detection Using Wait-For Graph

ALGORITHM Detect_Deadlock_WFG

INPUT:

Transactions T

Wait-For Graph G

OUTPUT:

Deadlock_Status

BEGIN

Create Wait-For Graph G.

```

FOR each transaction Ti DO
    FOR each transaction Tj DO
        IF Ti is waiting for a resource held by Tj THEN
            Add directed edge  $T_i \rightarrow T_j$  to G.
        END IF
    END FOR
END FOR

Perform DFS on G.

FOR each transaction Ti DO
    Mark Ti as VISITING.

    FOR each adjacent transaction Tj DO
        IF Tj is VISITING THEN
            RETURN "Deadlock Detected"
        END IF

        IF Tj is NOT VISITED THEN
            Perform DFS(Tj)
        END IF
    END FOR

    Mark Ti as VISITED.
END FOR

RETURN "No Deadlock"

```

END

Explanation:

A cycle in the Wait-For Graph indicates a deadlock. For example:

$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$

forms a cycle, so the transactions are deadlocked.

5. Basic Two-Phase Locking Protocol

ALGORITHM Two_Phase_Locking(Transaction T)

INPUT:

Transaction T

OUTPUT:

Transaction Result

BEGIN

Phase 1: Growing Phase

FOR each data item X required by T DO

Request appropriate lock on X.

IF lock is available THEN

Grant lock to T.

ELSE

Put T into WAIT state.

Wait until lock becomes available.

Grant lock to T.

END IF

END FOR

Continue transaction execution.

Phase 2: Shrinking Phase

When T releases its first lock:

Mark T as entering SHRINKING phase.

Release required locks.

DO NOT request any new lock.

IF transaction completes successfully THEN

Release all remaining locks.

COMMIT T.

ELSE

Release all locks.

ROLLBACK T.

END IF

END

Two Phases:

Growing Phase → Acquire Locks

Shrinking Phase → Release Locks

The main rule is that after a transaction releases a lock, it cannot acquire any new lock.

6. Timestamp-Based Concurrency Control Using Thomas' Write Rule

ALGORITHM Thomas_Write_Rule(T, X)

INPUT:

Transaction T

Data Item X

OUTPUT:

Action on Write Operation

BEGIN

$TS_T \leftarrow$ Timestamp of transaction T

$R_TS(X) \leftarrow$ Read Timestamp of X

$W_TS(X) \leftarrow$ Write Timestamp of X

IF $TS_T < R_TS(X)$ THEN


```

    Reject the write operation.
    Rollback transaction T.

ELSE IF TS_T < W_TS(X) THEN

    Ignore the write operation.
    Continue transaction T.

    // Thomas' Write Rule:
    // The write is obsolete because a newer transaction
    // has already written X.

ELSE

    Perform WRITE(X).

    W_TS(X) ← TS_T

END IF

END
Read Operation
ALGORITHM Timestamp_Read(T, X)

BEGIN

    IF TS_T < W_TS(X) THEN
        Reject READ.
        Rollback T.
    ELSE
        Perform READ(X).

        R_TS(X) ← MAX(R_TS(X), TS_T)
    END IF

END

END
Thomas' Write Rule:
If TS(T) < W_TS(X), the write is obsolete and can be ignored instead of aborting the transaction.

```

7. ARIES Recovery Algorithm

ALGORITHM ARIES_Recovery

INPUT:

Log
Database

OUTPUT:

Recovered_Database

BEGIN

/* PHASE 1: ANALYSIS */

Read the most recent checkpoint.

Initialize Transaction Table.
Initialize Dirty Page Table.

Scan the log forward from the checkpoint.

FOR each log record DO

 Update Transaction Table.

 IF log record modifies a page THEN
 Add page to Dirty Page Table if required.
 END IF

 Identify transactions that were active
 at the time of crash.

END FOR

/* PHASE 2: REDO */

Find the smallest Recovery LSN
from the Dirty Page Table.

Scan the log forward from the Recovery LSN.

FOR each update log record DO

 IF operation needs to be redone THEN
 Repeat the logged operation.
 END IF

END FOR

/* PHASE 3: UNDO */

Identify all loser transactions
that were incomplete at the time of crash.

WHILE loser transactions exist DO

 Select the next log record to undo.

 Undo the operation.

 Write a Compensation Log Record (CLR).

 Move to the previous log record
 of the transaction.

 IF transaction has reached its BEGIN record THEN
 Write END record.
 Remove transaction from loser list.
 END IF

END WHILE

Return recovered database.

END

ARIES Phases

Analysis → Redo → Undo

- Analysis: Determines active transactions and dirty pages.
- Redo: Repeats necessary logged operations.
- Undo: Rolls back incomplete transactions.

8. Shadow Paging Recovery

ALGORITHM Shadow_Paging

INPUT:

Database Pages

Update Operations

OUTPUT:

Committed Database

BEGIN

/* Initialization */

Create Shadow_Page_Table.

Copy Shadow_Page_Table to Current_Page_Table.

Set Current_Page_Table ← Shadow_Page_Table.

/* During Transaction */

FOR each update operation DO

Allocate a new page.

Write the updated data to the new page.

Modify Current_Page_Table
to point to the new page.

END FOR

/* Commit */

IF transaction completes successfully THEN

Flush all updated pages to disk.

Update Shadow_Page_Table
to point to Current_Page_Table.

Commit transaction.

ELSE

Current_Page_Table $\leftarrow$ Shadow_Page_Table.

Discard newly allocated pages.

Rollback transaction.

END IF

/* System Failure */

IF system failure occurs THEN

Current_Page_Table $\leftarrow$ Shadow_Page_Table.

Discard uncommitted changes.

Restore database using Shadow_Page_Table.

END IF

END

Main Idea:

Shadow Page Table $\rightarrow$ Stable Database

Current Page Table $\rightarrow$ Modified Database

If a failure occurs before commit, the shadow page table is used to restore the previous consistent state.

9. Immediate Update Recovery Using UNDO/REDO

ALGORITHM Immediate_Update_Recovery

INPUT:

Log

Database

OUTPUT:

Recovered Database

BEGIN

/* Normal Transaction Processing */

FOR each database update DO

Write log record:

$\langle T, X, \text{Old_Value}, \text{New_Value} \rangle$

Write log record to stable storage.

Update database page immediately.

END FOR

/* After System Crash */

Identify all transactions in the log.

Determine:

Winner Transactions = Transactions with COMMIT record

Loser Transactions = Transactions without COMMIT record

/* REDO Phase */

FOR each Winner Transaction T DO

FOR each update record of T in forward order DO

Write New_Value to database.

END FOR

END FOR

/* UNDO Phase */

FOR each Loser Transaction T DO

FOR each update record of T in reverse order DO

Write Old_Value to database.

END FOR

Write ABORT record for T.

END FOR

Flush recovered pages.

RETURN Recovered Database.

END

Recovery Rule

Committed Transaction → REDO

Uncommitted Transaction → UNDO

10. Database Connectivity Manager

ALGORITHM Database_Connectivity_Manager

INPUT:

Database Configuration

SQL Query

OUTPUT:

Query Result

BEGIN

/* Step 1: Initialize Connection Pool */

Create Connection Pool.

Set maximum number of connections.

Set minimum number of connections.

/* Step 2: Get Connection */

TRY

Connection ← Get connection from pool.

IF Connection is NULL THEN

Wait until a connection becomes available.

END IF

/* Step 3: Start Transaction */

Begin Transaction.

/* Step 4: Execute Query */

Prepare SQL Query.

Validate query parameters.

Execute SQL Query.

Store result in Result_Set.

/* Step 5: Commit */

IF execution is successful THEN

COMMIT Transaction.

Return Result_Set.

END IF

/* Step 6: Return Connection */

Return Connection to Connection Pool.

CATCH Database_Exception

/* Exception Handling */

ROLLBACK Transaction.

Log error message.

Return Connection to Connection Pool.

Display "Database Operation Failed."

FINALLY

Close unused resources.

Release connection if it is not already
returned to the pool.

END TRY

END

Connection Pool Flow

Request Connection



Get Connection from Pool



Begin Transaction



Execute Query

